Sorting Algorithms

1. Selection Sort

The selection sort algorithm sorts an array by repeatedly finding the minimum element (considering ascending order) from unsorted part and putting it at the beginning. The algorithm maintains two subarrays in a given array.

1) The subarray which is already sorted.

2) Remaining subarray which is unsorted.

In every iteration of selection sort, the minimum element (considering ascending order) from the unsorted subarray is picked and moved to the sorted subarray.

arr[] = 64 25 12 22 11

// Find the minimum element in arr[0...4]
// and place it at beginning

11 25 12 22 64

// Find the minimum element in arr[1...4]
// and place it at beginning of arr[1...4]
11 **12** 25 22 64

// Find the minimum element in arr[2...4]
// and place it at beginning of arr[2...4]
11 12 **22** 25 64

// Find the minimum element in arr[3...4]
// and place it at beginning of arr[3...4]
11 12 22 **25** 64

2. Bubble Sort

Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in wrong order.

		0	1	2	3	4	5	6	7
i = 0	j								
	0	5	3	1	9	8	2	4	7
	1	3	5	1	9	8	2	4	7
	2	3	1	5	9	8	2	4	7
	3	3	1	5	9	8	2	4	7
	4	3	1	5	8	9	2	4	7
	5	3	1	5	8	2	9	4	7
	6	3	1	5	8	2	4	9	7
i=1	0	3	1	5	8	2	4	7	9
	1	1	3	5	8	2	4	7	
	2	1	3	5	8	2	4	7	
	3	1	3	5	8	2	4	7	
	4	1	3	5	2	8	4	7	
	5	1	3	5	2	4	8	7	
i=2	0	1	3	5	2	4	7	8	
	1	1	3	5	2	4	7		
	2	1	3	5	2	4	7		
	3	1	3	2	5	4	7		
	4	1	3	2	4	5	7		
i=3	0	1	3	2	4	5	7		
	1	1	3	2	4	5			
	2	1	2	3	4	5			
	3	1	2	3	4	5			
i=4	0	1	2	3	4	5			
	1	1	2	3	4				
	2	1	2	3	4				
i=5	0	1	2	3	4				
	1	1	2	3					
i=6	0	1	2	3					
		1	2						

First Pass:

(**5 1** 4 2 8) –> (**1 5** 4 2 8), Here, algorithm compares the first two elements, and swaps since 5 > 1.

(1 **5 4** 2 8) –> (1 **4 5** 2 8), Swap since 5 > 4

(1 4 **5 2** 8) –> (1 4 **2 5** 8), Swap since 5 > 2

(1 4 2 **5 8**) –> (1 4 2 **5 8**), Now, since these elements are already in order (8 > 5), algorithm does not swap them.

Second Pass:

(**1 4** 2 5 8) –> (**1 4** 2 5 8)

(1 **4 2** 5 8) –> (1 **2 4** 5 8), Swap since 4 > 2

(1 2 **4 5** 8) –> (1 2 **4 5** 8)

(1 2 4 **5 8**) –> (1 2 4 **5 8**)

Now, the array is already sorted, but our algorithm does not know if it is completed. The algorithm needs one **whole** pass without **any** swap to know it is sorted.

Third Pass:

(**1 2** 4 5 8) –> (**1 2** 4 5 8)

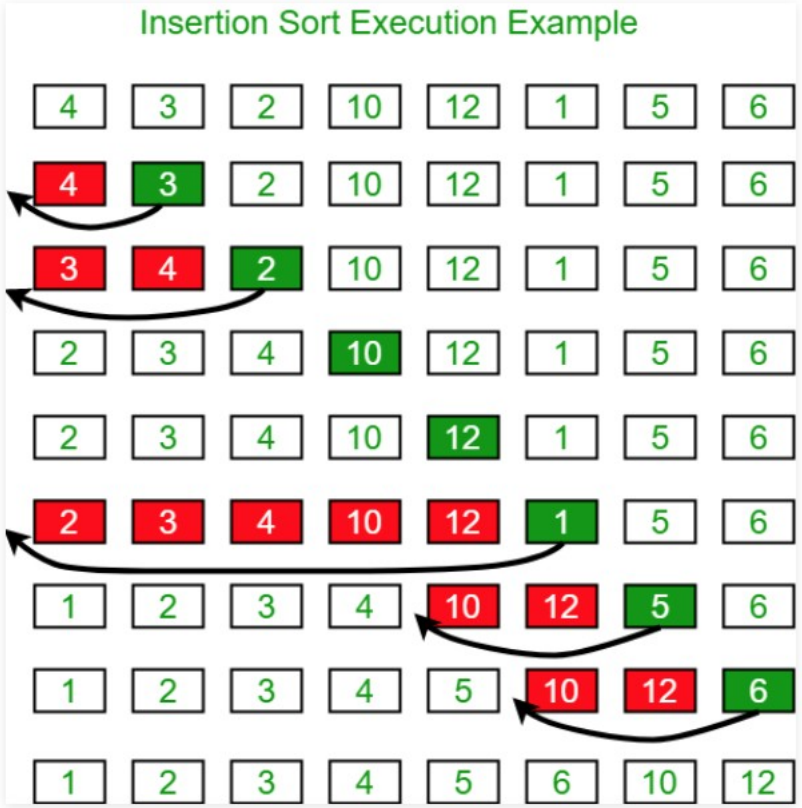
(1 **2 4** 5 8) –> (1 **2 4** 5 8)

(1 2 **4 5** 8) –> (1 2 **4 5** 8)

(1 2 4 **5 8**) –> (1 2 4 **5 8**)

3. Insertion Sort

Insertion sort is a simple sorting algorithm that works similar to the way you sort playing cards in your hands. The array is virtually split into a sorted and an unsorted part. Values from the unsorted part are picked and placed at the correct position in the sorted part.



Algorithm

To sort an array of size n in ascending order:
1: Iterate from arr[1] to arr[n] over the array.
2: Compare the current element (key) to its predecessor.
3: If the key element is smaller than its predecessor, compare it to the elements before. Move the greater elements one position up to make space for the swapped element.

4. Shell Sort

- ***Shell sort*** is a sorting algorithm that treats the input as a collection of interleaved lists and sorts each list individually with a variant of the insertion sort algorithm.
  - Shell sort uses ***gap values*** to determine the number of interleaved lists.
  - A ***gap value*** is a positive integer representing the distance between elements in an interleaved list.
- **Procedure**:

1. Choose a gap value K
2. Sort K interleaved lists using insertion sort
3. Sort the entire list using insertion sort

Worst time: O(N^3/2)
Stable: no
In-place: yes

5. Quick Sort

- ***Quicksort*** is a sorting algorithm that repeatedly *partitions* the input into low and high parts (each part unsorted), and then *recursively* sorts each of those parts.
  - To partition the input, quicksort chooses a ***pivot*** to divide the data into low and high parts.
  - The ***pivot*** can be any value within the array being sorted, commonly the value of the middle array element.
- ***Quicksort*** is often the default sorting algorithm used by programming languages' libraries.
  - E.g Java - `Arrays.sort();`

5. Quick Sort

1. The partitioning algorithm uses two index variables **`low`** and **`high`**, initialized to the left and right sides of the current elements being sorted.
2. If the value at index **`low`** is less than the ***pivot*** value, the algorithm increments **`low`**, because the element should remain in the low partition.
3. Likewise, if the value at index **`high`** is greater than the ***pivot*** value, the algorithm decrements **`high`**, because the element should remain in the high partition.
4. Then, if **`low`** >= **`high`**, all elements have been partitioned, and the partitioning algorithm *returns* **`high`**, which is the index of the last element in the **`low`** partition.
5. Otherwise, the elements at indices **`low`** and **`high`** are *swapped* to move those elements to the correct partitions.
6. Finally, the algorithm then increments **`low`**, decrements **`high`**, and repeats.

5. Quick Sort

```
int partition(numbers, left, right) {

    //Pick middle value as pivot
    midpoint = left + (right - left) /
2
    pivot = numbers[midpoint]

    low = left
    high = right

    done = false
    while (!done) {
        while (numbers[low] < pivot) {
            low++
        }
        //continue
        while (pivot < numbers[h]) {
            high--
        }
        if (low >= h) {
            done = true
        } else {

swap(numbers[low],numbers[high])
            low++
            high--
        }
    }
    return high
}
```

5. Quick Sort

- Once partitioned, each partition needs to be sorted.
- ***Quicksort*** is typically implemented as a recursive algorithm using calls to quicksort to sort the low and high partitions.
- This recursive sorting process continues until a partition has one or zero elements, and thus already sorted.

5. Quick Sort

- Best-case: $O(Nlog(N))$
- Average-case: $O(Nlog(N))$
- Worst-case: $O(N^2)$
  - List is sorted
  - List is sorted in reverse order
  - All elements are the same
- Stable: no
- In-place: yes

```
Quicksort(numbers, left, right) {
   /* Base case: If 1 or zero elements,
      partition is already sorted */
   if (left >= right) {
      return
   }

   /* Partition the array.
      Value mid is the location of last
      element in low partition. */
   int mid = Partition(numbers, left,
right)

   /* Recursively sort low and high
       partitions */
   Quicksort(numbers, left, mid)
   Quicksort(numbers, mid + 1, right)
}
```

6. Merge Sort

- ***Merge sort*** is a sorting algorithm that divides a list into two halves, recursively sorts each half, and then merges the sorted halves to produce a sorted list. The recursive partitioning continues until a list of 1 element is reached, as list of 1 element is already sorted.
- **Procedure**

1. MergeSort recursively divides the list into two halves.
2. The list is divided until a list of 1 element is found.
3. A list of 1 element is already sorted.
4. At each level, the sorted lists are merged together while maintaining the sorted order.

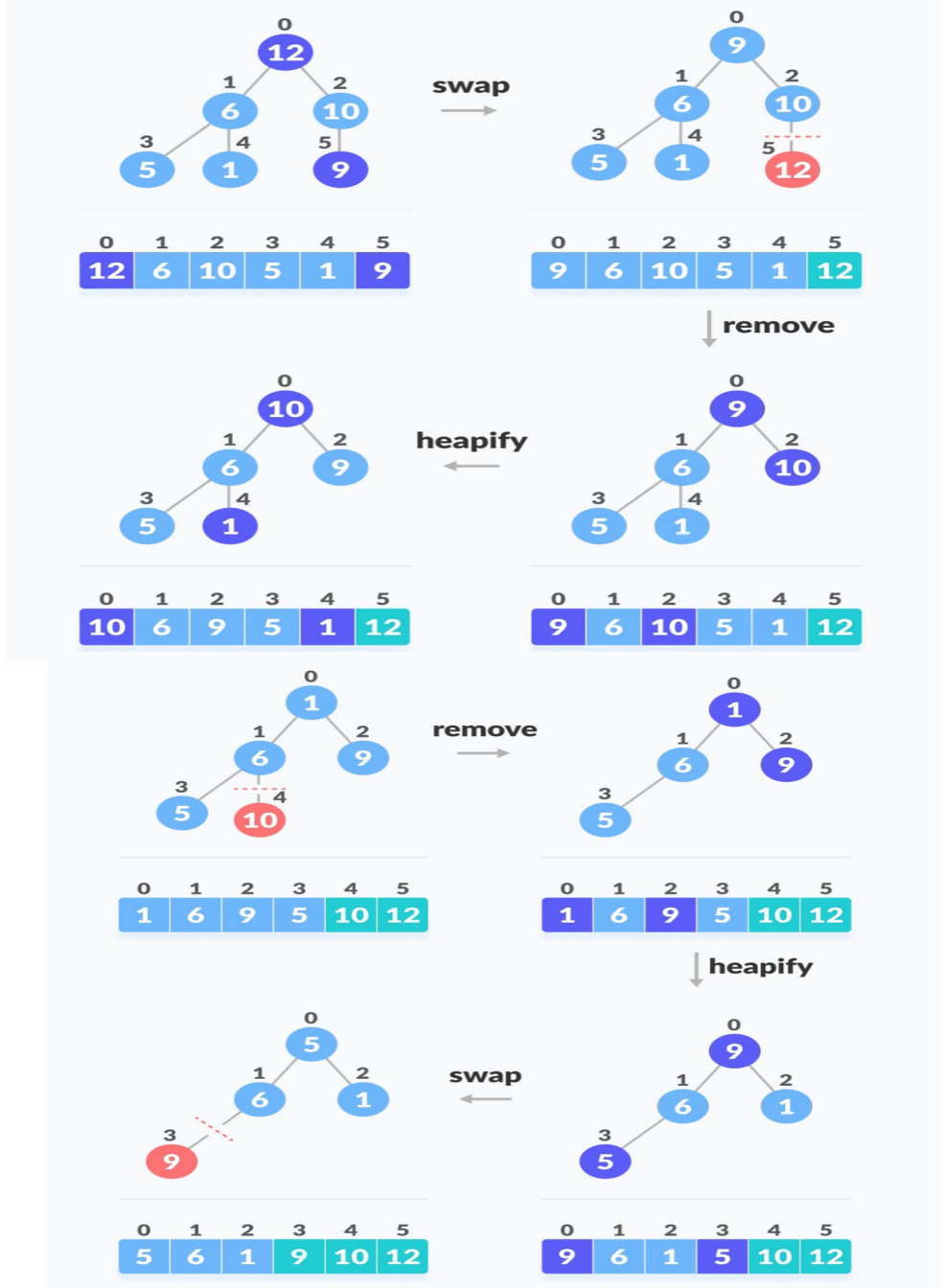
6. Merge Sort

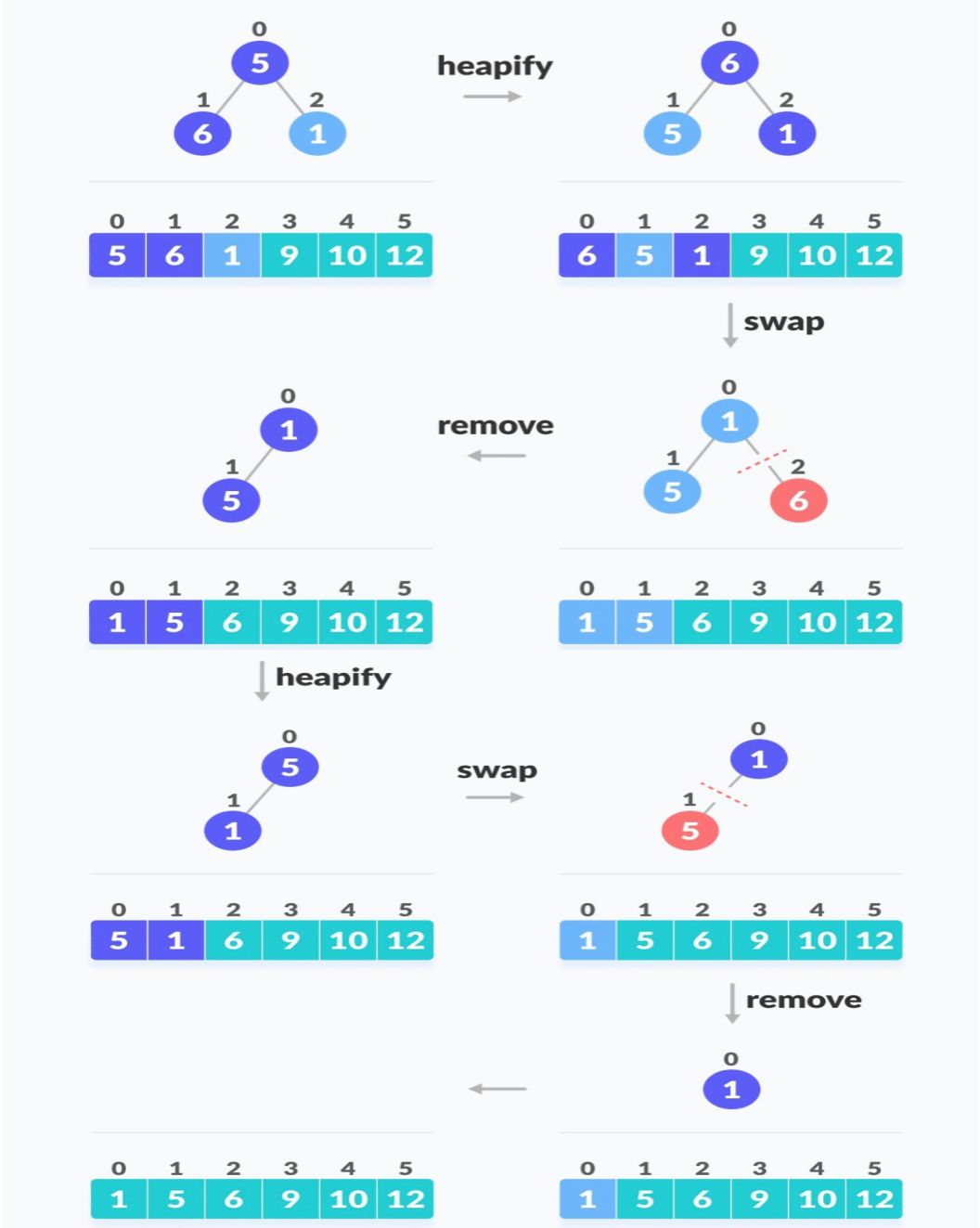
- Best-case: $O(Nlog(N))$
- Average-case: $O(Nlog(N))$
- Worst-case:$O(Nlog(N))$
- Stable: yes
- In-place: no – requires extra space

```
MergeSort(arr[], l,  r)
If r > l
     1. Find the middle point to divide the array
into two halves:
               middle m = (l+r)/2
     2. Call mergeSort for first half:
               Call mergeSort(arr, l, m)
     3. Call mergeSort for second half:
               Call mergeSort(arr, m+1, r)
     4. Merge the two halves sorted in step 2 and
3:
               Call merge(arr, l, m, r)
```

7. Heap Sort

1. Since the tree satisfies Max-Heap property, then the largest item is stored at the root node.
2. **Swap:** Remove the root element and put at the end of the array (nth position) Put the last item of the tree (heap) at the vacant place.
3. **Remove:** Reduce the size of the heap by 1.
4. **Heapify:** Heapify the root element again so that we have the highest element at root.
5. The process is repeated until all the items of the list are sorted.


swap
remove
heapify
remove
heapify
swap


heapify
swap
remove
heapify
swap
remove

7. Heap Sort

Heap Sort Complexity

Time Complexity	
Best	O(nlog n)
Worst	O(nlog n)
Average	O(nlog n)
Space Complexity	O(1)
Stability	No

Sorting Algorithm

Algorithm	*Worst*	*In place? (Yes/No)*	*Stable? (Yes/No)*
Bubble			
Selection			
Insertion			
Shell*			
Merge			
Quick			
Heap			

In-Class Activity (Possible interview Q)

Sort an array of 0's, 1's, and 2's (Dutch National Flag Problem)

For example,

Input: $\{0, 1, 2, 2, 1, 0, 0, 2, 0, 1, 1, 0\}$

Output: $\{0, 0, 0, 0, 0, 1, 1, 1, 1, 2, 2, 2\}$

Selection, insertion and bubble sort all have a complexity of O(N^2). Heap and merge have O(N log N)

Given that the array just has 0,1 and 2's. Can we do better than that ??

Design a solution with complexity O(N).

Solution A: Counting Sort

```java
int[] A = {0, 2, 1, 0, 1, 2, 1, 2, 2, 1, 0, 1, 0, 2, 0};
int zero = 0;
int one = 0;
int two = 0;

for (int i = 0; i < A.length; i++) {
    if (A[i] == 0)
        zero++;
    else if (A[i] == 1)
        one++;
    else
        two++;
}

for (int i = 0; i < A.length; i++) {
    if (zero > 0) {
        A[i] = 0;
        zero--;
    } else if (one > 0) {
        A[i] = 1;
        one--;
    } else {
        A[i] = 2;
        two--;
    }
}
```

Time Complexity: O(n+k) where n is the number of elements in input array and k is the range of input.

When n>>k
Time complexity O(N)

However, this requires two traversals of the array.

Solution B: Three-way-partition

We can rearrange the array in a single traversal using an alternative **linear-time partition routine** that separates the values into three groups:

- The values less than the pivot,
- The values equal to the pivot, and
- The values greater than the pivot.

Time complexity: O(N) and requires one traversal through array.

```
// Linear time partition routine to sort an array containing 0, 1, and 2.
// It is similar to 3-way partitioning for the Dutch national flag problem.
public static void threeWayPartition(int[] A)
{
    int start = 0, mid = 0;
    int pivot = 1;
    int end = A.length - 1;

    while (mid <= end)
    {
        if (A[mid] < pivot)          // current element is 0
        {
            swap(A, start, mid);
            ++start;
            ++mid;
        }
        else if (A[mid] > pivot)     // current element is 2
        {
            swap(A, mid, end);
            --end;
        }
        else {                       // current element is 1
            ++mid;
        }
    }
}
```